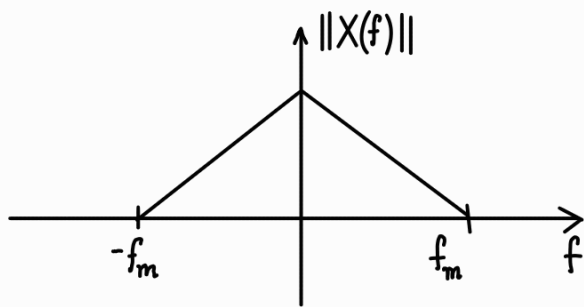
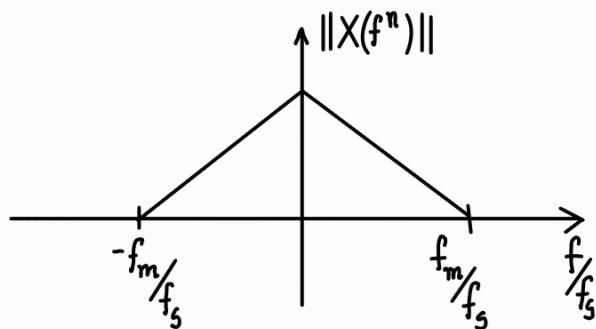


Sampling and aliasing (revisited)

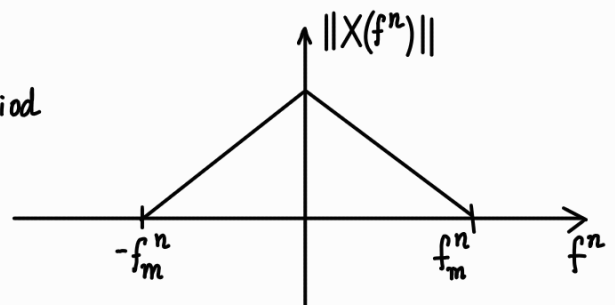
Spectrum of a continuous signal after CTFT be —



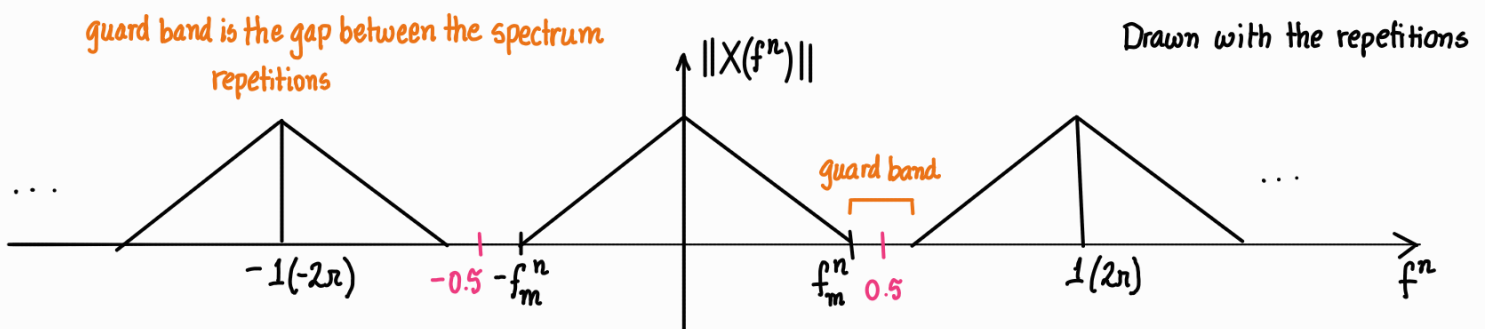
When this signal is sampled the frequency becomes normalized —



Drawn for 1 period

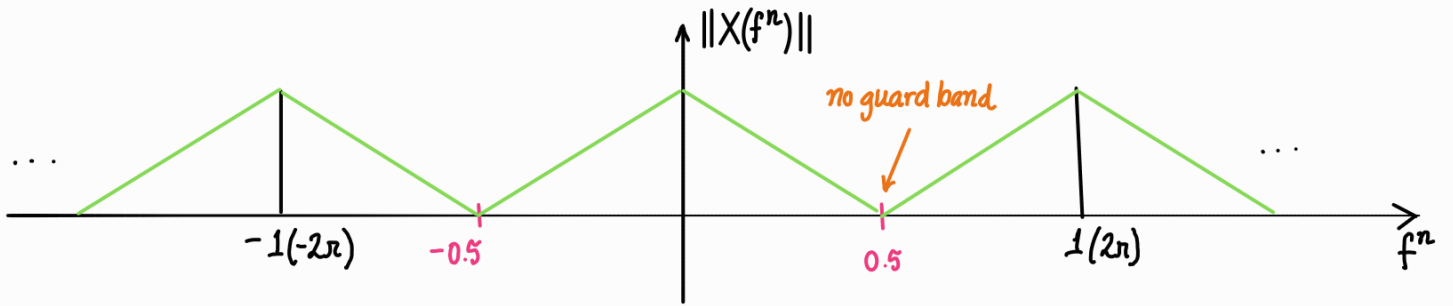


Now we have two points of interest at -0.5 and 0.5 . Let's set them on the graph assuming $\frac{f_m}{f_s} < 0.5$.

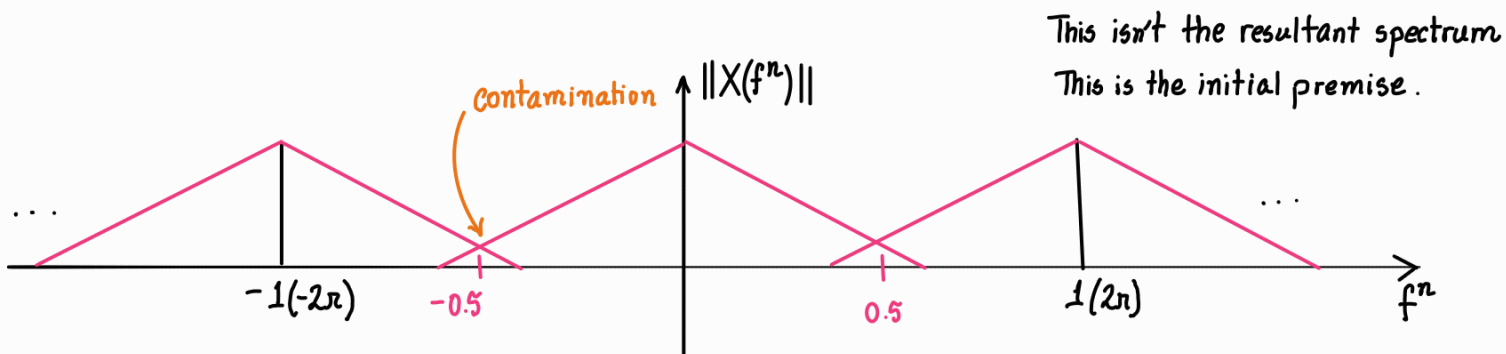


This how the spectrum looks when the signal is properly sampled.

When sampled at Nyquist rate —

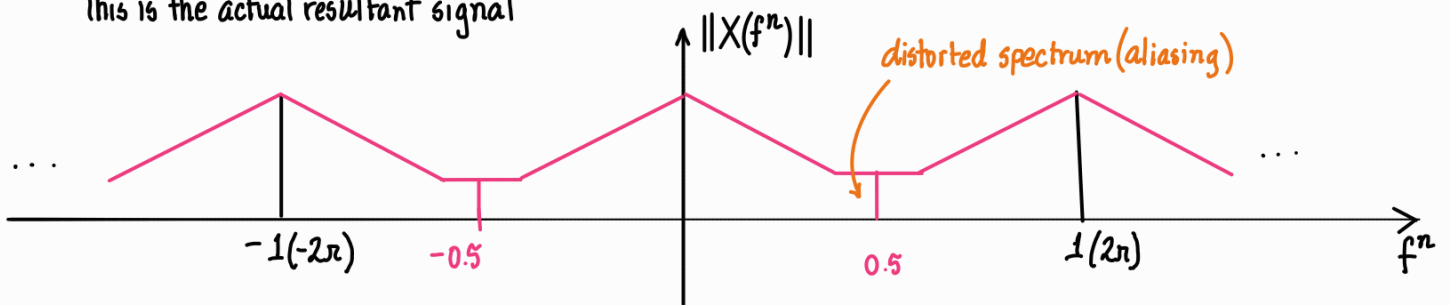


When sampled below Nyquist rate —



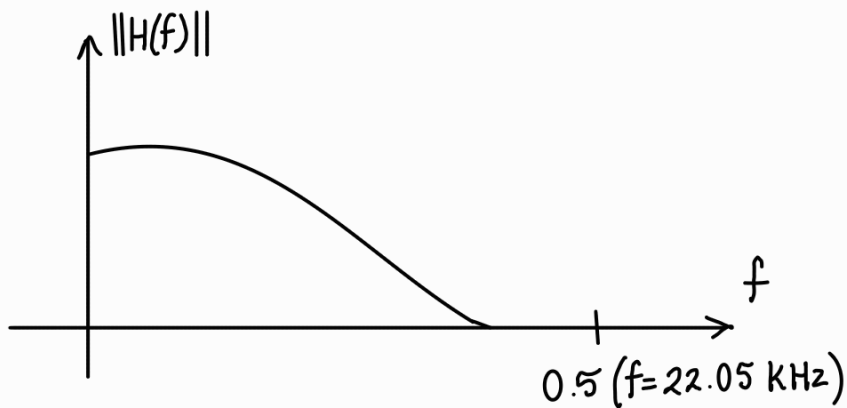
This isn't the resultant spectrum
This is the initial premise.

This is the actual resultant signal

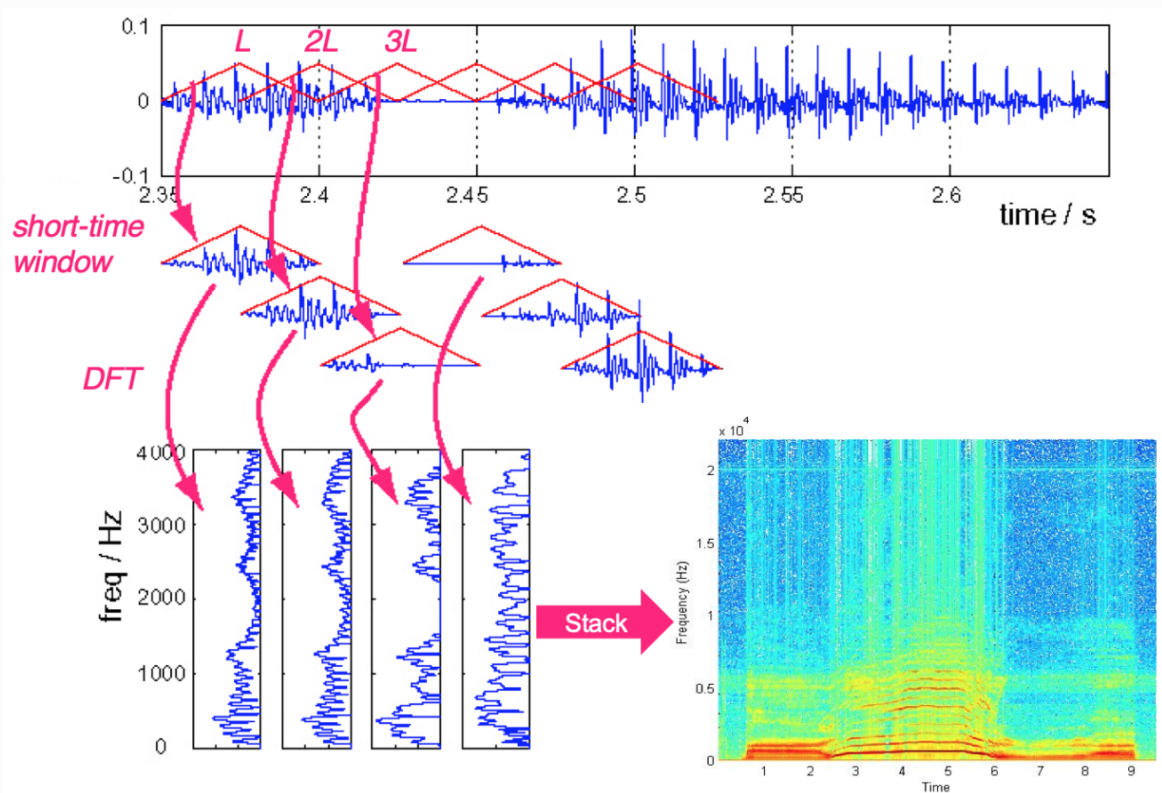


Temporal info loss in FT and STFT

We have extensively studied all types of Fourier transforms. We now know how to convert a time domain signal to its frequency domain. But consider a music signal. Doing FT on the music signal would probably give you a spectrum that looks something like —



This okay. But with the flow change in music, would it not better if we could retain the time info too? We could do something like —



This is called Short-Time Fourier Transform or STFT

$$\text{STFT}\{x(t)\} = X(\tau, \omega) = \int_{\tau - \text{len}(\omega)/2}^{\tau + \text{len}(\omega)/2} x(t) \cdot w(t - \tau) e^{-j\omega t} dt \quad \leftarrow \text{continuous variant}$$

time shift value of the window centre τ $\rightarrow$ sliding window $\rightarrow$ integrate only over the window

$$\text{STFT}\{x[n]\} = X[m, k] = \sum_{n=m - \text{len}(\omega)/2}^{m + \text{len}(\omega)/2} x[n] \cdot w[n - m] e^{-jk\Omega_0 n}$$

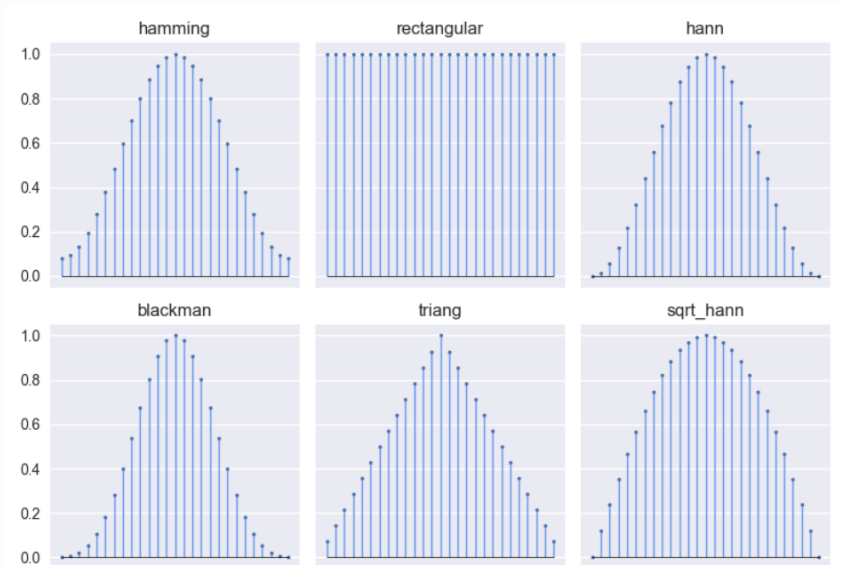
We generally iterate over the signal and just take the FFT.

The inverse of STFT is called ISTFT and is a bit complex. It will be discussed at the end.

Now for the window functions, they can be changed. Rectangular window is probably the most naive way to approach STFT. When I work with STFT, I generally use Hann or Blackman.

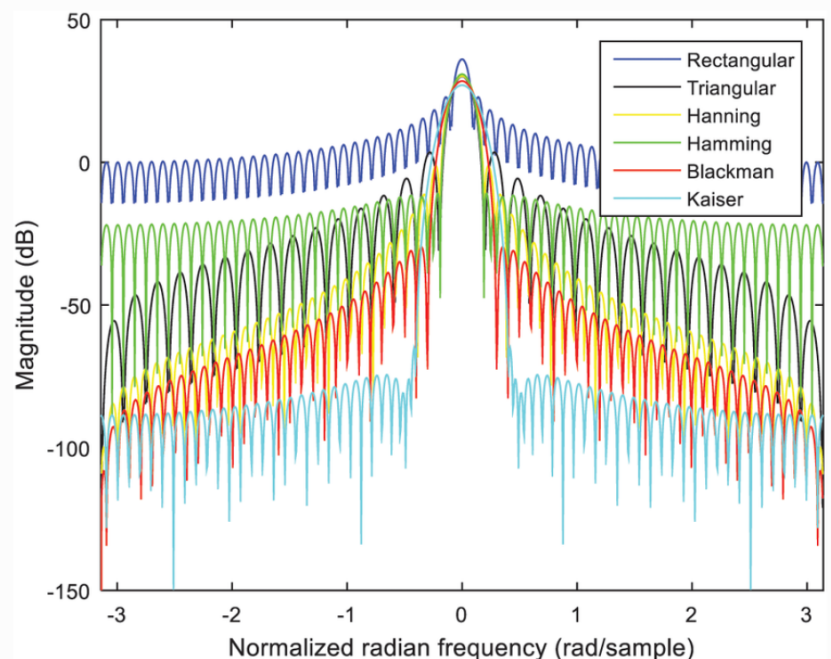
Why even use different windows?

Rectangular windowing creates discontinuity in time domain which produces unwanted ripples in frequency spectrum known as spectral leakage. The tapering windows minimize the spectral leakage.



Do note that all windowing functions may not perform equally on all signals. Also generally, a high attenuation in the stop band may be associated with a slower roll off.

$$\text{magnitude (dB)} = 10 \log_{10} \left(\frac{V}{V_{\text{ref}}} \right)$$



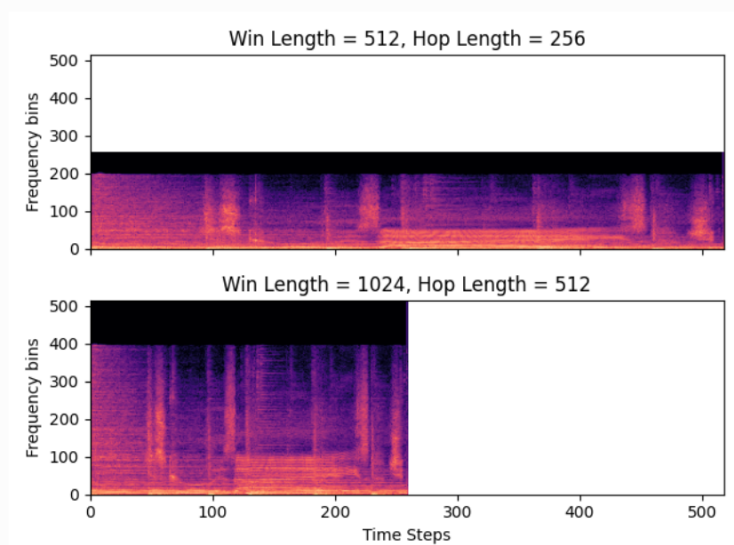
Things to consider while doing STFT —

Since we will be doing DFT, the frequency resolution will be limited by the window size. But the window cannot be arbitrarily increased as it will limit the amount of time steps we get before we reach end of the signal. So for STFT we have a time frequency resolution trade off.

Can we improve our technique to increase the frequency bins without decreasing time resolution? We can separate the DFT window and the sliding window.

So say window size is 512. Instead of applying DFT on 512 samples, we take DFT on 1024 samples by zero-padding the windowed signal by 256 on both sides. This is just the DFT becoming more like

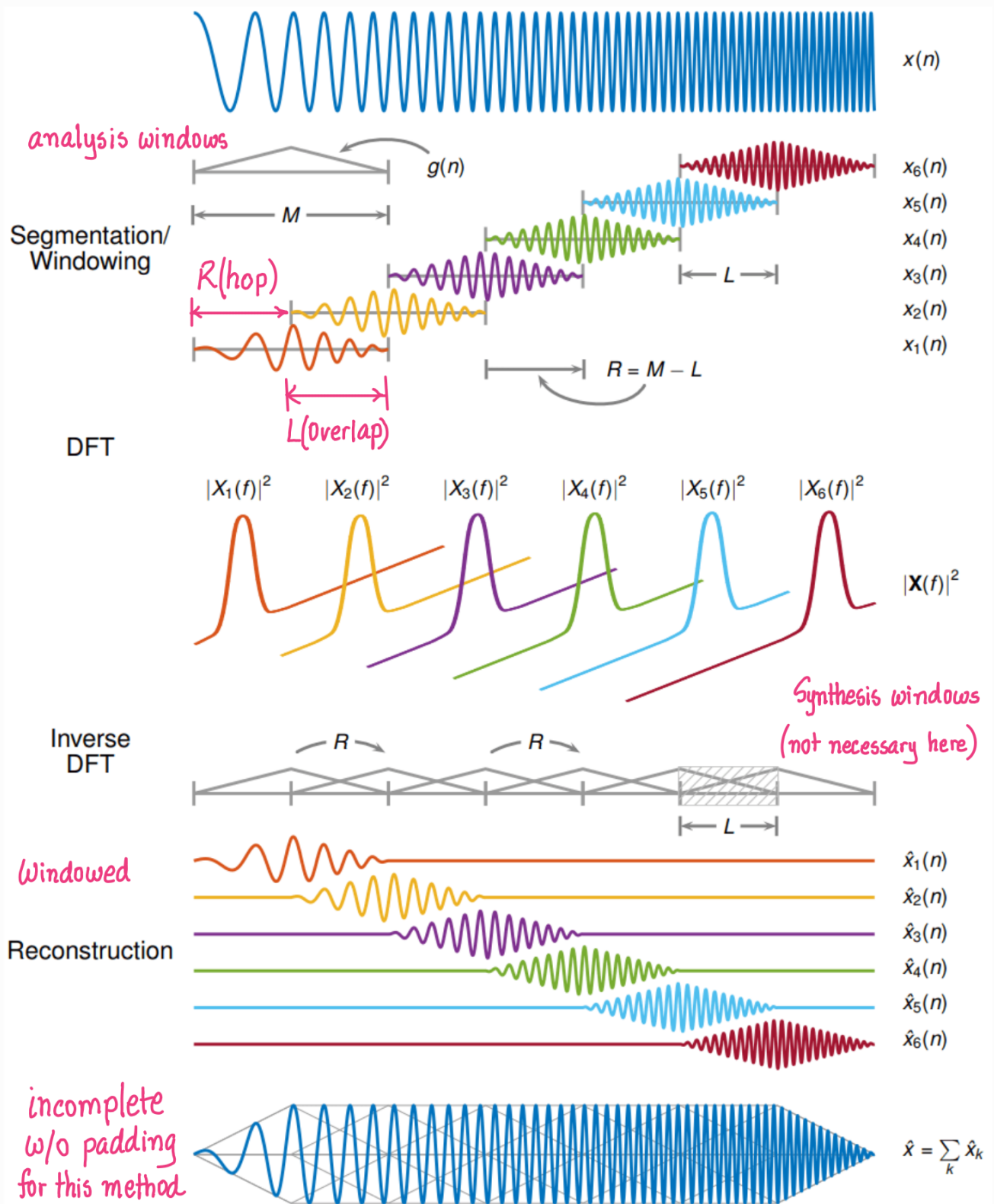
the DTFT.



Can we improve the time resolution without compromising the frequency resolution?

Yes we can add some overlap. This is generally implemented by setting a hop size (basically stride) less than the window size. So 50% overlap is basically hop size equal to half the window size. Generally we do not allow too much overlapping as it increases the computation cost a lot.

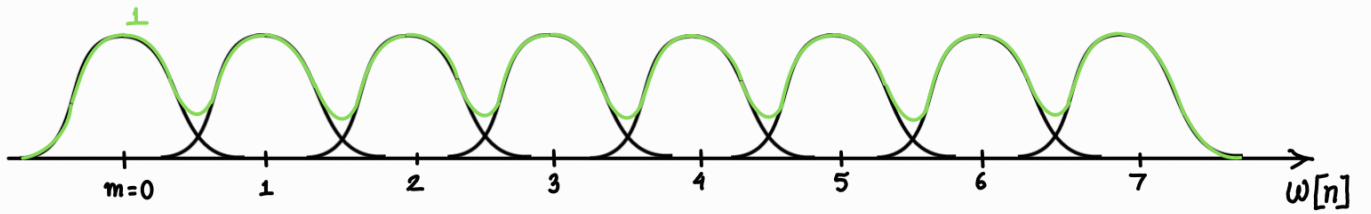
Inverse STFT —



This method is called **Overlap Add (OLA)**. This is too naive and doesn't allow frequency manipulation. We need **Weighted OLA (WOLA)** for general inverse. Synthesis windows makes sense under frequency manipulation.

Note — Synthesis windows are generally the same as the analysis windows.

In the OLA example, the summation of the window 'train' was 1 which requires very specific hop size for specific windows. That is quite restricting. Lets take a Hann window for example at an odd overlap that doesn't result in $\sum_m \omega_m[n-mH] = \omega[n] = 1 \forall n$. H is the hop size.



We need to correct this compression somehow. Lets go through the formulation to figure out how we can fix this.

Windowing, $\omega_A(x[n]) = x[n] \cdot \omega_A[n-mH]$ where $m \in \mathbb{Z}^+$ and $m_{\max} = \left\lfloor \frac{N-L+2P}{H} \right\rfloor$

FFT at each $m \rightarrow X[m, K] = \sum_{n=-L/2}^{L/2} x[n] \cdot \omega_A[n-mH] e^{-j\Omega_0 K n} \quad \forall m$

Manipulation in frequency domain $\rightarrow \hat{X}[m, K] = \sum_{n=-L/2}^{L/2} \hat{x}[n] \cdot \omega_A[n-mH] e^{-j\Omega_0 K n} \quad \forall m$

→ assuming no change to the window function by accounting everything in $\hat{x}[n]$

IFFT at each $m \rightarrow \hat{y}[n] = \hat{x}[n] \cdot \omega_A[n-mH]$

Windowing, $\omega_s(\hat{y}[n]) = \hat{x}[n] \cdot \omega_A[n-mH] \cdot \omega_s[n-mH]$

→ can contain sharp components on the edges because of the manipulation in the frequency domain

Generally $\omega_A = \omega_s$. Then, $\omega_s(\hat{y}[n]) = \hat{x}[n] \cdot \omega^2[n-mH]$

Then applying OLA $\rightarrow \hat{x}[n] \cdot \sum_m \omega^2[n-mH]$

$$\therefore \hat{x}[n] = \frac{\hat{x}[n] \sum_m \omega^2[n-mH]}{\sum_m \omega^2[n-mH]} = \frac{\hat{x}[n] \cdot \omega^2[n]}{\omega^2[n]} \longrightarrow \text{the weight for WOLA}$$

The formulation may feel different at times. Just try understanding what is going on in the bigger picture. You can write equations yourself if you understand the concept.

For all intent and purpose, we will consider the windows to be centered with padding = $L//2$ to be present unless otherwise mentioned.

For dimension formula go through convolution. The formula is the same in both cases.

Note — The formula we derived doesn't actually work according to theory because w_H does get changed. But most libraries do use this implementation. It gives you an estimated $\hat{x}_n$.